

Team: Macintosh

Registration Number: 24BCI0194

Objective: To instrument the xv6 kernel to log user-space system calls by implementing a new `trace(int on)` system call.

Implementation Steps:

Process Structure (`proc.h`): Added the `int trace_on`; integer flag to the struct `proc` definition to

track whether tracing is enabled for a given process.

Process Lifecycle (`proc.c`): Updated `allocproc()` to initialize tracing to zero (`p->trace_on = 0`;) for

all new processes. Modified `fork()` to ensure child processes inherit the tracing state from their

parent (`np->trace_on = curproc->trace_on`;) .

System Call Definition (`syscall.h`): Assigned a unique identifier for the new system call by adding

`#define SYS_trace 22`.

System Call Handler (`sysproc.c`): Implemented the `sys_trace()` function. This function uses `argint()` to read the user-provided argument and assigns that value to `curproc->trace_on`, effectively toggling the trace state.

Dispatcher Hook and Logging (`syscall.c`): Added `sys_trace` to the system call function pointer array. Created a `syscall_names` string array to map `syscall` integers to readable names. Finally, updated the `syscall()` dispatcher to check `curproc->trace_on`; if true, it uses `cprintf` to log the

process ID, the system call name, and the return value stored in the `eax` register.

User Space Exposure (`user.h` & `usys.S`): Made the system call accessible to user-space programs by declaring the prototype `int trace(int)`; in `user.h` and adding the `SYSCALL(trace)` macro in `usys.S`.

Testing and Verification (`echo.c`): Injected a `trace(1)`; function call at the beginning of the `echo`

program's `main()` function.

Execution: Compiled the kernel using `make clean` and `make qemu`. Running the command `echo hello` successfully produced the expected system call trace logs alongside the standard output, confirming the implementation.